

```
=====
                        INSERTION SORT ALGORITHM
                        in C Language
=====
```

```
PROGRAM DESCRIPTION:
```

Insertion Sort is an efficient sorting algorithm for small datasets. It builds the sorted array one element at a time by inserting each element into its correct position within the already-sorted portion of the array.

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

int main() {
    int arr[100], n, i, j, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Insertion Sort
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
=====
                        PROGRAM EXPLANATION
=====
```

1. ARRAY DECLARATION AND INPUT:
 - Declares array of maximum 100 elements
 - Takes number of elements from user
 - Reads all elements
2. OUTER LOOP (Building Sorted Portion):

```
for (i = 1; i < n; i++)
```

 - Starts from second element (index 1)
 - First element is already sorted
 - `key = arr[i]` stores current element to be inserted

3. INNER LOOP (Finding Insertion Position):
 - while (j >= 0 && arr[j] > key)
 - Searches from i-1 towards the beginning
 - Shifts elements greater than key to the right
 - Stops when correct position is found
4. INSERTION:
 - arr[j + 1] = key
 - Places key in its correct position
 - Array maintains sorted order up to index i
5. OUTPUT:
 - Prints sorted array

```
=====
                        ALGORITHM STEPS
=====
```

Insertion Sort Process:

- Step 1: Start with second element (i = 1)
Step 2: For each element from index 1 to n-1:
- a) Save element as key
 - b) Compare key with sorted elements (from i-1 to 0)
 - c) Shift elements greater than key one position right
 - d) Insert key in correct position
 - e) Move to next element

Key Idea:

At each step, array[0...i] is sorted, and we insert arr[i+1].

```
=====
                        EXAMPLE WALKTHROUGH
=====
```

Array: [64, 34, 25, 12, 22]
Goal: Sort in ascending order

Initial State:

Sorted: [64] Unsorted: [34, 25, 12, 22]

Pass 1 (i=1, key=34):

Compare 34 with sorted portion [64]
64 > 34, shift 64 to right
Array: [64, 64, 25, 12, 22]
Insert 34 at position 0
Array: [34, 64, 25, 12, 22]
Sorted: [34, 64] Unsorted: [25, 12, 22]

Pass 2 (i=2, key=25):

Compare 25 with sorted portion [34, 64]
64 > 25, shift 64 to right: [34, 64, 64, 12, 22]
34 > 25, shift 34 to right: [34, 34, 64, 12, 22]
Insert 25 at position 0
Array: [25, 34, 64, 12, 22]
Sorted: [25, 34, 64] Unsorted: [12, 22]

Pass 3 (i=3, key=12):

Compare 12 with sorted portion [25, 34, 64]
64 > 12, shift: [25, 34, 64, 64, 22]
34 > 12, shift: [25, 34, 34, 64, 22]
25 > 12, shift: [25, 25, 34, 64, 22]
Insert 12 at position 0
Array: [12, 25, 34, 64, 22]

Sorted: [12, 25, 34, 64] Unsorted: [22]

Pass 4 (i=4, key=22):

Compare 22 with sorted portion [12, 25, 34, 64]

64 > 22, shift: [12, 25, 34, 64, 64]

34 > 22, shift: [12, 25, 34, 34, 64]

25 > 22, shift: [12, 25, 25, 34, 64]

12 < 22, insert at position 2

Array: [12, 22, 25, 34, 64]

Sorted: [12, 22, 25, 34, 64]

Final Sorted Array: [12, 22, 25, 34, 64]

COMPLEXITY ANALYSIS

Time Complexity:

Best Case: $O(n)$ - Already sorted, 1 comparison per element

Average Case: $O(n^2)$ - Random order

Worst Case: $O(n^2)$ - Reverse sorted, i comparisons for element i

Detailed Analysis:

Best Case (Sorted Array):

Comparisons: $n - 1$

Shifts: 0

Total: $O(n)$

Worst Case (Reverse Sorted):

Comparisons: $1 + 2 + \dots + (n-1) = n(n-1)/2 = O(n^2)$

Shifts: Similar to comparisons

Total: $O(n^2)$

Average Case:

Comparisons: $n(n-1)/4 \approx O(n^2)$

Shifts: $n(n-1)/4 \approx O(n^2)$

Total: $O(n^2)$

Comparison Table:

Algorithm	Best	Average	Worst	Space
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

ADVANTAGES & DISADVANTAGES

Advantages:

• Efficient for small datasets ($n \leq 100$ elements)

• $O(n)$ best case (already sorted)

• No extra space required (in-place)

• Stable sorting (preserves order of equal elements)

• Online algorithm (can sort as data arrives)

• Adaptive (efficient for nearly sorted data)

• Simple to implement

• Good cache performance

Disadvantages:

- $O(n^2)$ time complexity (inefficient for large datasets)
- Not suitable for very large datasets
- More complex than Selection Sort
- Slower than QuickSort or MergeSort
- Many element shifts in worst case

KEY CHARACTERISTICS

Stability:

Stable - Elements with equal values maintain relative order

Example: [3a, 1, 3b, 2]

After sorting: [1, 2, 3a, 3b] (3a and 3b maintain order)

In-Place:

Yes - Requires only $O(1)$ extra space

Comparison-Based:

Yes - Uses comparisons between elements

Adaptive:

Yes - $O(n)$ for nearly sorted data, $O(n^2)$ for reverse sorted

Online:

Yes - Can process elements one at a time

Natural:

Yes - Works well with linked lists

PRACTICAL APPLICATIONS

Ideal For:

1. Small datasets (< 100 elements)
2. Partially sorted or nearly sorted data
3. Online sorting (data arrives sequentially)
4. Linked lists (better than QuickSort/MergeSort)
5. Situations with limited memory
6. When stability is required
7. When minimizing writes is important

Used In:

1. Java's Arrays.sort() for small arrays
2. Python's sorted() and sort() for small runs
3. C++ std::sort() as part of hybrid algorithm
4. Database indexing
5. Adaptive sorting algorithms

Not Ideal For:

1. Large datasets (> 1000 elements)
2. Real-time systems with strict time limits
3. Data with expensive comparisons (should use better algorithm)

OPTIMIZATION TECHNIQUES

1. Binary Insertion Sort:

- Use binary search to find insertion position
- Reduces comparisons to $O(n \log n)$

- Shifts remain $O(n^2)$
- Total: Still $O(n^2)$ but fewer comparisons

Modified Code:

```

// Find position using binary search
int left = 0, right = i - 1;
while (left <= right) {
    int mid = (left + right) / 2;
    if (arr[mid] <= key) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
// Now left is insertion position

```

2. Hybrid Approach:

- Use Insertion Sort for small arrays (< 32)
- Switch to QuickSort for larger arrays

3. Timsort (Python's Sorting):

- Divides array into small chunks
- Sorts each chunk with Insertion Sort
- Merges sorted chunks

4. Optimized Shift:

- Use memmove() for array shifting
- Faster than individual element moves

COMPARISON: INSERTION vs SELECTION vs BUBBLE

Insertion Sort vs Selection Sort:

```

â€œ Insertion is adaptive ( $O(n)$  for sorted data)
â€œ Selection is not adaptive (always  $O(n^2)$ )
â€œ Insertion is stable
â€œ Selection is not stable
â€œ Insertion better for nearly sorted data
â€œ Selection uses fewer swaps

```

Insertion Sort vs Bubble Sort:

```

â€œ Both have  $O(n)$  best case
â€œ Insertion has  $O(n^2)$  average case
â€œ Bubble always  $O(n^2)$  average case
â€œ Insertion is generally faster
â€œ Both are stable and in-place
â€œ Insertion better for most cases

```

Insertion Sort vs QuickSort:

```

â€œ QuickSort faster for large datasets
â€œ QuickSort  $O(n \log n)$  average case
â€œ Insertion  $O(n^2)$  average case
â€œ Insertion better for small datasets (< 100)
â€œ Insertion  $O(n)$  for nearly sorted
â€œ Used together: QuickSort + Insertion for hybrid

```

CODE VARIATIONS

Variation 1: With Comparison Count

```

int comparisons = 0;
for (i = 1; i < n; i++) {
    key = arr[i];
    j = i - 1;
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
        comparisons++;
    }
    arr[j + 1] = key;
}

```

Variation 2: Descending Order

```

    while (j >= 0 && arr[j] < key) { // Changed > to <
        arr[j + 1] = arr[j];
        j--;
    }

```

Variation 3: With Early Exit

```

    if (key == arr[i - 1]) continue; // Already in place

```

=====